OPERATING SYSTEMS PROGRAMMING ASSIGNMENT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Semaphore and Monitor Implementations of Readers-Writers and Dining Philosophers

Student Name: M Akanksha Bhat
Register Number: NNM24IS115
Institution: NMAM Institute of Technology, Nitte
Submitted To: Dr Jason Elroy Martis

GitHub Link: https://github.com/Akanksha-bhat/OS-Synchronization-Assignment

1. Project Overview

There may be several threads active concurrently in an OS. If they are not being correctly synchronized, failures can arise, leading to wrong data, clashes, and starvation, in some cases. In order to avoid these problems, various synchronization mechanisms such as semaphores and monitors have been proposed.

To implement the current work, two types of synchronization problems are taken into account: the Readers-Writers Problem and the Dining Philosophers Problem. Python programming language is used for this purpose. To make the implementation easier to comprehend, HTML documents are generated that illustrate the way processes change their state and use shared resources.

A very attractive feature of this project is that the simulation interface interacts with the running program. Events that are logged while the code is in execution in real life are used to generate HTML output that visualizes the process and explains how synchronization should really work.

2. Readers-Writers – Semaphore Solution

The Readers-Writers dilemma involves controlling access to some common information, for example a file or database. Several Readers can access this information simultaneously but the Writer can do so only when no one else accesses this information.

The Used Semaphores:

- **turnstile** - Bans threads from entering and gives the Writers who are waiting in the queue light at the entrance to the resource.
- **resource** - Manages how the common information is being accessed.
- **count_mutex** - Prevents the reader_count variable from being changed simultaneously by several threads.
- **reader_count** - The number of Readers that are reading at a given point in time.
- **shared_data** - The data with the value of 0 at the start.

Execution Logic:

Several Readers can read the information at the same time. The first Reader locks the resource so that the Writer cannot change the information. The last Reader unlocks the resource.

The Writer waits for the resource to be freed and it gets access to this information. Afterwards, the Writer increases shared_data by 10 and releases the resource.

The program also creates an interactive HTML page that reflects what threads are doing at each point (waiting, active, finished), as well as a log

3. Readers-Writers - Monitor Solution

The Readers-Writers problem is related to managing access to shared data, like a file or data base. Multiple Readers can read the data concurrently, but a Writer needs a solo access for making modifications.

Monitor Components:

• **threading. Condition()** – It allows threads to wait till they can fulfill conditions and access the common resource.

• **active_readers** – It keeps the record of number of Readers that are reading.

• **waiting_readers and waiting_writers** – To keep records of Readers and Writers that are waiting to perform their actions.

• **active_writer** – to determine if there is a Writer at the moment.

• **shared_data** – It keeps the overall common value which is 0 at the beginning.

Execution Logic:

In case a Reader arrives the first check is if there is a Writer currently writing or waiting. If no, the Reader can read the data in the company of the Readers. Once the last Reader has done reading all the waiting threads are informed.

A Writer should wait in case there are any Readers working with the data or another Writer is working at the moment. Only then can be Writer get access to the data and adds 10; After writing is done.

4. Dining Philosophers – Semaphore Solution

In the problem of the Dining Philosophers, the four philosophers must share four forks while they are eating. Each philosopher thinks for a while and then waits for both forks in order to eat. The semaphores are used in the program to control the access to the forks and help avoid deadlocking situations.

Components Used:

- **forks** – Semaphores that control access to each fork and prevent two philosophers from using the same fork at the same time.
- **room** – Allows a maximum of three philosophers to compete for forks at once.

- **FairWaiter** – Manages requests in order to give philosophers a fair chance to get the forks.
- **states** – Keeps track of what each philosopher is doing, such as thinking, waiting, or eating.
- **meals** – Counts the number of meals completed by each philosopher.

Execution Logic:

Each philosopher thinks first and then waits for their turn to get the forks. The philosopher picks up both forks in a fixed order and starts eating. After finishing the meal, they release both forks so other philosophers can use them.

The program also creates an interactive HTML page that displays the philosophers' activities and fork usage. It includes play, pause, reset, speed, and timeline controls to make it easier to follow the execution.

5. Dining Philosophers – Monitor Solution

The Dining Philosophers monitor scheme allows four philosophers to utilize four forks without causing any conflicts. The monitor observes the action of each of the philosophers and which forks are in use. A philosopher is able to commence eating only when both forks they need are available and it is their turn to eat.

Components Used:

- **threading.Condition()** – Allows philosophers to wait until the required conditions are satisfied and notifies waiting threads when the situation changes.
- **states** – Tracks whether each philosopher is thinking, hungry, or eating.
- **forks** – Keeps track of which philosopher is using each fork.
- **waiting** – Maintains the order of philosophers waiting for their turn.
- **meals** – Counts how many meals each philosopher has completed.

Execution Logic:

Each philosopher starts by thinking and then becomes hungry when they are ready to eat. They wait until both neighbouring forks are available and their turn arrives. Once they get both forks, they begin eating. After finishing, they release the forks and notify the other waiting philosophers so they can check whether they can proceed.

The program also generates an interactive HTML page showing the philosophers' activities and fork usage. Play, pause, reset, speed, and timeline controls allow the execution to be viewed step by step.

6. Event Recording and HTML Generation

1. Event Recording

• The program records events while the Python threads are running.

• It notes the time the thread or philosopher ID, the current state and how resources are used.

• Locks and conditions protect the recorded data when many threads are operating.

2. Svg Visualization

• Once all threads finish the recorded events are turned into JSON. Inserted into an HTML page.

• JavaScript uses this data to display each step of the execution.

• The Readers-Writers simulation displays how readers and writers share the resource.

• The Dining Philosophers simulation shows the philosophers seated at the table and which forks they hold.

3. Interactive Controls

• Play/Pause: starts or pauses the simulation.

• Reset: returns the visualization to the start.

• Speed: lets users adjust the playback speed.

• Timeline: lets users scroll through the recorded events.

7. Execution Traces for All Four Implementations

Tracing the program helps us see how Readers, Writers, and Philosophers use shared resources. The events and timestamps show an example of how the program works. Since threads may run in different orders each time, the sequence of events can also change from one run to another.

7.1. Readers-Writers – Semaphore Solution

This trace shows that multiple readers can read the shared data at the same time, while the writer waits for the readers to finish.

0.001s R1 -> waiting to read

0.001s R1 -> starts reading (value = 0)

0.052s R2 -> waiting to read

0.052s R2 -> starts reading (value = 0)

0.122s W1 -> waiting to write

0.122s W1 -> waiting for the resource

0.180s R1 -> finishes reading

0.220s R2 -> finishes reading

0.221s W1 -> starts writing (value: 0 -> 10)

0.300s W1 -> finishes writing

Result: Both readers can read together. Once they finish, the writer gets access and increases shared_data by 10.

7.2. Readers-Writers – Monitor Solution

This trace shows how the monitor controls access using a condition variable. Readers can read together, but new readers must wait when a writer is already waiting.

0.001s R1 -> arrives and starts reading (value = 0)

0.052s R2 -> arrives and starts reading (value = 0)

0.122s W1 -> waits for access

0.202s R3 -> waits because a writer is waiting

0.250s R1 -> finishes reading

0.300s R2 -> finishes reading

0.301s W1 -> starts writing (value: 0 -> 10)

0.400s W1 -> finishes writing

0.401s R3 -> starts reading (value = 10)

Result: The monitor makes sure the writer waits until the active readers finish. After writing is complete, the waiting reader can access the updated data.

7.3. Dining Philosophers – Semaphore Solution

The above demonstrations show the philosophers and how they wait for getting their chance to pick two forks and eat their food. The code allows only three philosophers to compete for the forks and the fixed order of picking the forks eliminates the possibility of deadlock.

0.150s P0 -> gets hungry

0.180s P1 -> gets hungry

0.200s P0 -> picks up forks F0 and F1

0.201s P0 -> starts eating

0.250s P2 -> gets hungry

0.300s P1 -> waits for forks

0.350s P2 -> picks up forks F2 and F3

0.351s P2 -> starts eating

0.600s P0 -> finishes eating

0.601s P0 -> puts down forks F1 and F0

0.650s P2 -> finishes eating

0.651s P2 -> puts down forks F3 and F2

0.652s P1 -> gets the available forks

0.653s P1 -> starts eating

Result: A philosopher can start eating only after acquiring both forks. Semaphores control access to the forks and help prevent deadlock, ensuring that all philosophers can share the forks safely and properly.

7.4. Dining Philosophers – Monitor Solution

This trace shows how the monitor checks fork availability and lets philosophers take turns using a FIFO waiting queue.

0.120s P0 -> is thinking

0.150s P0 -> gets hungry and waits

0.151s P0 -> picks up forks F0 and F1

0.152s P0 -> starts eating

0.180s P1 -> gets hungry and waits

0.200s P2 -> gets hungry and waits

0.250s P0 -> puts down forks F0 and F1

0.251s P0 -> finishes eating

0.252s P1 -> picks up forks F1 and F2

0.253s P1 -> starts eating

0.500s P1 -> puts down forks F1 and F2

0.501s P1 -> finishes eating

0.502s P2 -> picks up forks F2 and F3

0.503s P2 -> starts eating

Result: The monitor allows a philosopher to eat only when both forks are free. After a philosopher releases the forks, waiting philosophers are notified and can check if they are able to eat.

8. Race Condition, Mutual Exclusion, Deadlock and Starvation

1. Race Condition

A race condition occurs when two or more threads attempt to access the same data concurrently. This may result in conflicts. Synchronization prevents race conditions and makes sure that the shared data remains secure in the case of Readers-Writers problem. The shared data is essential. The shared data should be secured. The shared data should be managed properly.

2. Mutual Exclusion

Mutual exclusion makes sure that only one thread or process accesses a shared resource at a time when needed. In the Readers-Writers problem, only one writer can modify the shared data at a time. In the Dining Philosophers problem, only one philosopher can use a fork at a time. This helps avoid conflicts and ensures that the program runs safely and correctly.

3. Deadlock

Deadlock occurs when threads are stuck waiting for resources. In the case of the Dining Philosophers problem the forks have a certain order in the solution, so that threads do not enter deadlock. In the solution of the monitor the philosopher waits until both forks are available. It is necessary to take care of the resources. The resources should not be neglected.

4. Starvation

Starvation happens when a thread has to wait too long because other threads keep getting access to the resource first. In the Dining Philosophers monitor solution, a FIFO queue helps philosophers get their turns in order. In the Readers-Writers solutions, synchronization manages access to shared data and helps reduce unnecessary waiting.

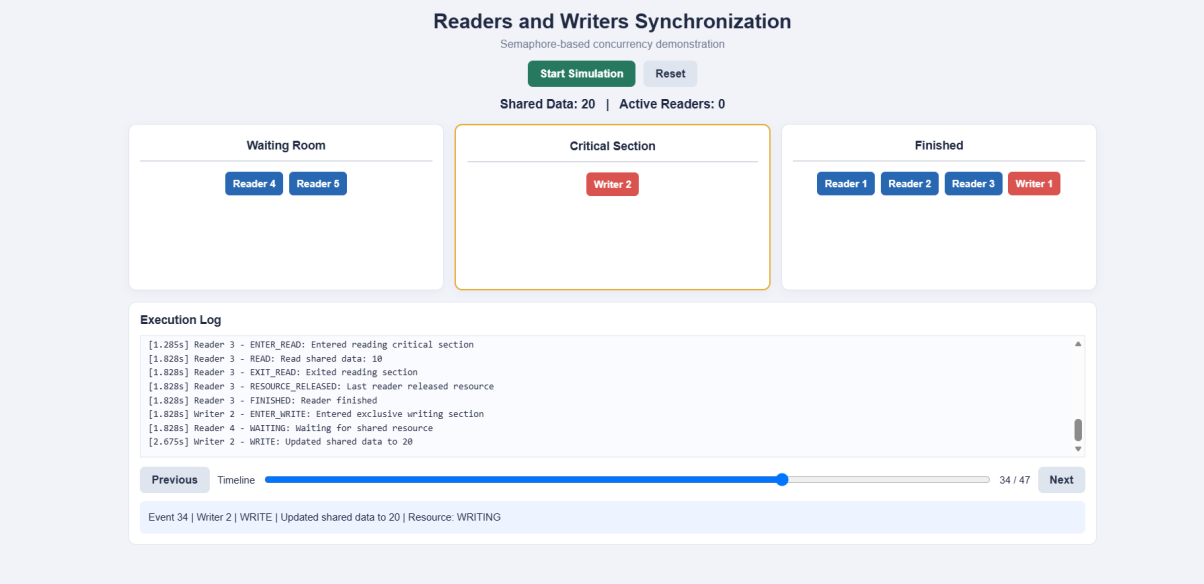
9. Semaphore vs Monitor Comparison

Semaphore	**Monitor**
Uses wait() and signal() to control resource access.	Uses methods and condition variables to manage resource access.
Requires careful handling by the programmer.	Keeps shared data and synchronization logic together.
Controls how many threads can access a resource.	Allows access based on specified conditions.
Incorrect use can cause synchronization problems.	Makes synchronization easier to organize and manage.
In Dining Philosophers, controls access to forks.	Allows a philosopher to eat when both forks are available and it is their turn.
In Readers-Writers, controls access to shared data.	Uses conditions to manage Readers and Writers.

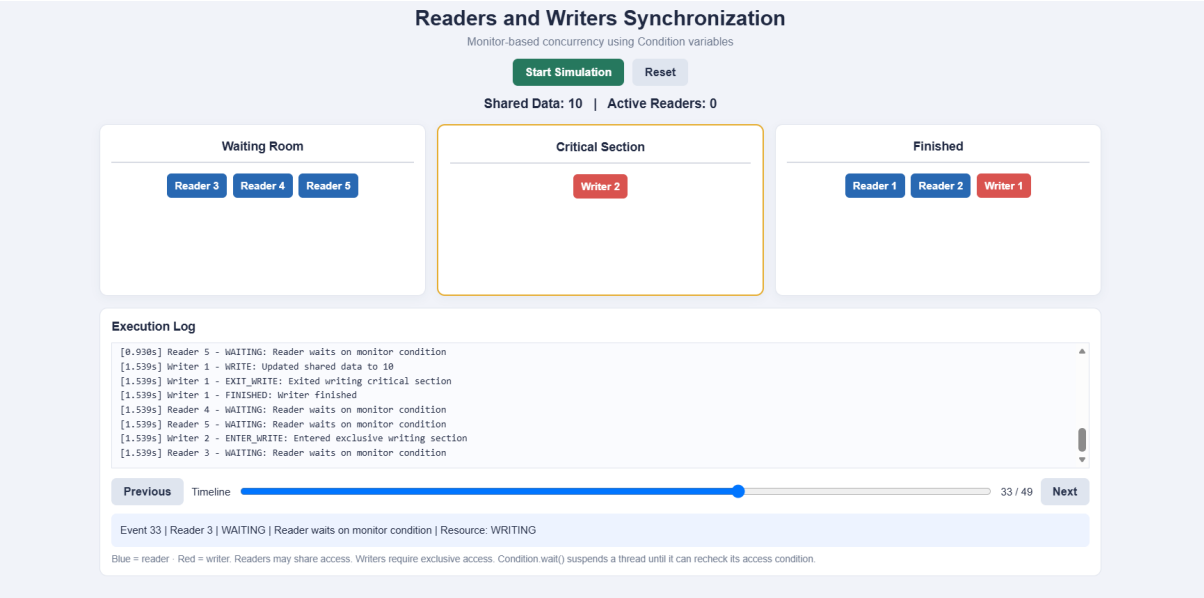
To summarize: both semaphores and monitors help threads to share resources safely and avoid resource conflicts while at the same time semaphores provide the programmer with direct means of control of thread synchronization through basic synchronization primitives whereas monitors make the task of code management simpler and easier.

10. Screenshots of Generated HTML Simulations

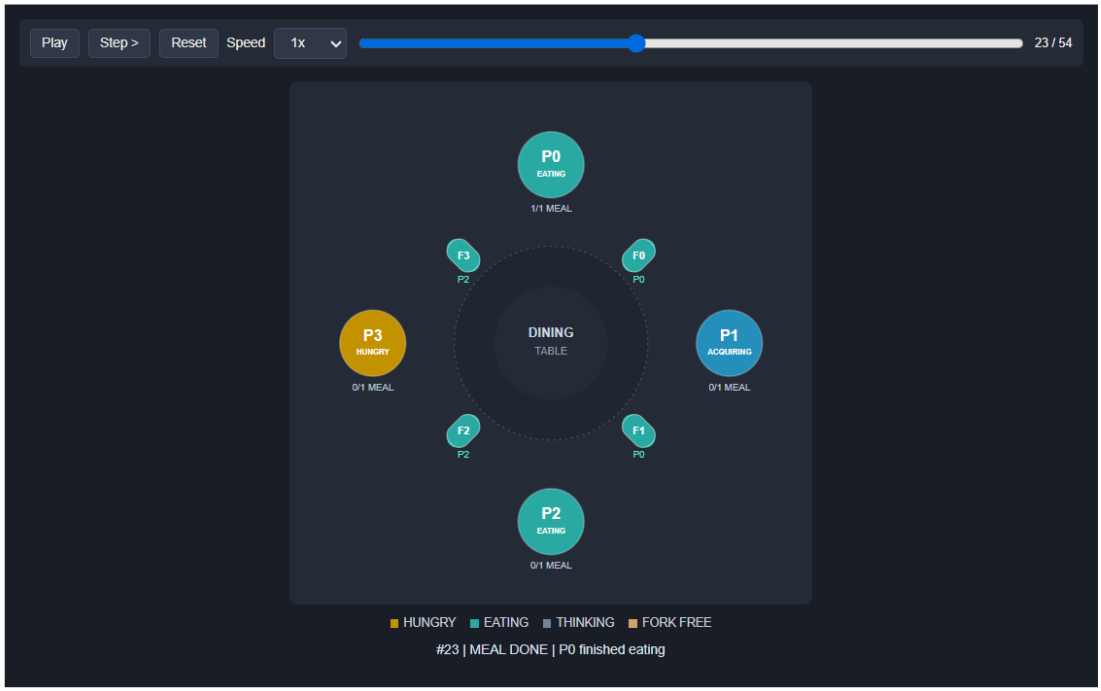
10.1 Readers-Writers Using Semaphore:



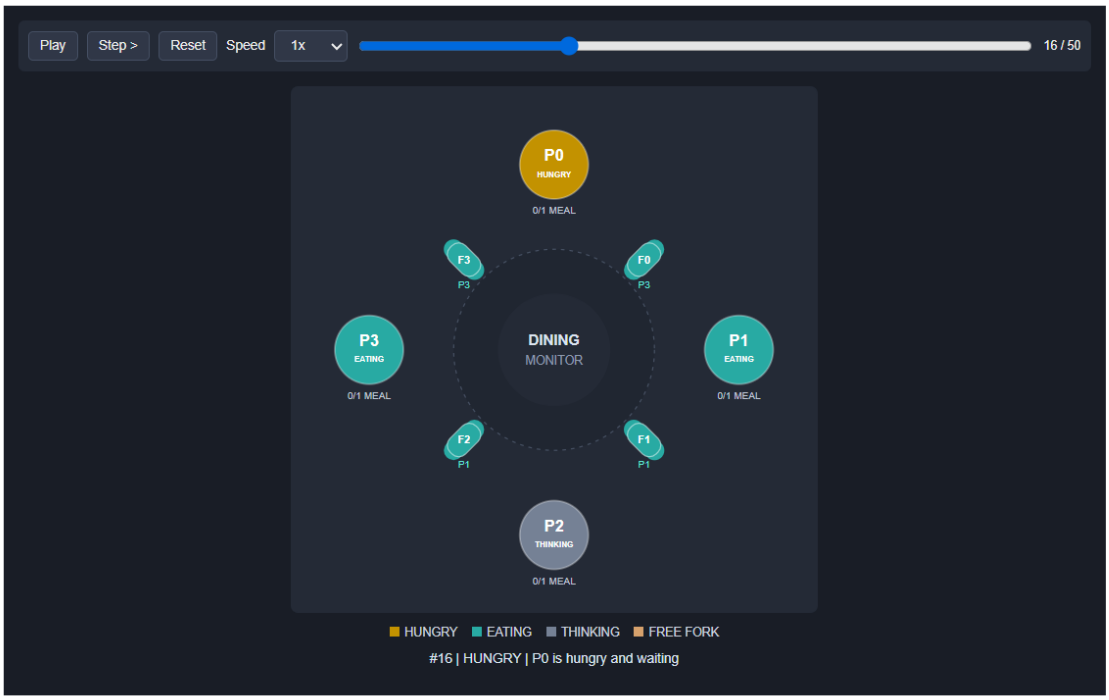
10.2 Readers-Writers Using Monitors:



10.3 Dining Philosophers Using Semaphore:



10.4 Dining Philosophers Using Monitor:



11. AI Use Summary

AI systems were employed in this project for planning, writing the basic code for all the four problems, managing synchronization between threads, solving problems, and designing the HTML/JavaScript visualization dashboard.

My contribution was mostly about launching the programs, analyzing their effectiveness, correcting problems, and verifying the coincidence of the visualization and the real program running. An example of my work here is solving the problem with the setup when I ran the files inside the ZIP folder, which proved to be problematic and inaccessible. I also worked on making the HTML visualization available immediately after any running program finishes its work.

When verifying the solution of the Dining Philosophers problem, I noticed that some lines in the SVG structure were inconsistent with the actual positions of forks. This allowed me to conclude that it is necessary to check the program's logic and its visual representation together.

I also studied the way semaphores, locks, and condition variables help in thread management in the Readers-Writers problem and in the Dining Philosophers as well. I learned here that Python condition variables do not guarantee FIFO scheduling by themselves.

All in all, AI made the working process easier.

12. References

1. **Python Software Foundation.** *Python Documentation – Threading.* Used to understand threads, locks, semaphores, and condition variables. https://docs.python.org/3/library/threading.html
2. **Python Software Foundation.** *Python Documentation – Queue.* Reference for queue-based management of waiting threads. https://docs.python.org/3/library/queue.html
3. **MDN Web Docs.** *JavaScript Guide.* Reference for JavaScript concepts used in the interactive visualization dashboard. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
4. **MDN Web Docs.** *SVG: Scalable Vector Graphics.* Reference for drawing and positioning visual elements in the Dining Philosophers interface. https://developer.mozilla.org/en-US/docs/Web/SVG
5. **Operating Systems Course Materials.** Concepts related to process synchronization, mutual exclusion, race conditions, deadlock, starvation, semaphores, and monitors.
6. **AI-Assisted Development.** AI tools were used as supporting resources for initial code development, debugging, and visualization design.

13. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Name: M Akanksha Bhat

USN: NNM24IS115

Signature:



Date: 14/10/2026